

# Matplotlib & Seaborn

Drawing charts so you can SEE your data

AI/ML Engineer Learning Series · Deep Dive Edition

|            |                                                          |
|------------|----------------------------------------------------------|
| Topic      | Matplotlib & Seaborn                                     |
| Matplotlib | The basic drawing tool — full control over every line    |
| Seaborn    | Built on Matplotlib — makes pretty charts with less code |
| Best For   | Looking at data, finding patterns (EDA), reports         |
| Used In    | Every data project — right after cleaning with Pandas    |

## What's Inside

---

Here is everything covered in this guide, in order:

| #  | Section             | What you'll learn                          |
|----|---------------------|--------------------------------------------|
| 1  | Why Draw Charts?    | What these tools are and why they matter   |
| 2  | Parts of a Chart    | Figure, axes, labels — the building blocks |
| 3  | Line Plot           | Show trends and changes over time          |
| 4  | Bar Plot            | Compare different groups                   |
| 5  | Scatter Plot        | See how two things relate                  |
| 6  | Histogram           | See how values are spread out              |
| 7  | Making It Pretty    | Titles, colors, labels, legends            |
| 8  | Many Charts at Once | Subplots side by side                      |
| 9  | Seaborn             | Prettier charts with much less code        |
| 10 | Boxplot & Heatmap   | Powerful charts for finding patterns       |
| 11 | Which Chart to Use? | Pick the right chart for the job           |
| 12 | Real ML Example     | Full EDA on a dataset                      |
| 13 | Common Mistakes     | What to avoid                              |
| ★  | Cheatsheet          | Quick reference of every command           |

# 1. Why Draw Charts?

Numbers in a table are hard to understand. If I show you 1000 numbers, you can't tell what's going on. But if I draw those numbers as a chart, you see the pattern in one second. That is the whole point of these tools.

**Matplotlib** is the main drawing tool in Python. It can draw any chart you want, and you control every tiny detail. **Seaborn** sits on top of Matplotlib. It makes good-looking charts with much less code, and it works perfectly with Pandas tables.

In Machine Learning, you use these all the time during **EDA** (Exploratory Data Analysis — **Visualize, Understand, and Communicate**). Before you build a model, you draw charts to understand your data.

■ *Simple way to remember: Matplotlib = full control but more code. Seaborn = pretty and quick but less control. Most people use both together.*

# 2. The Parts of a Chart

Every Matplotlib chart has two main parts. The **Figure** (**Canvas** — **Figure**) is the whole drawing area, like a blank page. The **Axes** (**Axes** — **Figure**) is one actual chart inside that page. One figure can hold many axes (many small charts).

| Part         | What it is             | Plain meaning                         |
|--------------|------------------------|---------------------------------------|
| Figure       | fig — the whole canvas | The full page                         |
| Axes         | ax — one plot area     | One single chart                      |
| Title        | ax.set_title()         | The name on top of the chart          |
| X-axis label | ax.set_xlabel()        | What the bottom line means            |
| Y-axis label | ax.set_ylabel()        | What the side line means              |
| Legend       | ax.legend()            | Small box telling which color is what |
| Grid         | ax.grid()              | Light background lines to read values |

The normal way to start any chart:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4)) # make a page + one chart
ax.plot([1,2,3], [4,5,6])             # draw on the chart
ax.set_title('My First Chart')         # add a title
plt.show()                             # show it on screen

# To save instead of show:
plt.savefig('mychart.png', dpi=150, bbox_inches='tight')
```

# 3. Line Plot — Show Trends

A **line plot** connects points with a line. It is best when you want to show how something changes over time or in order — like temperature through the day, or your model's accuracy over training rounds.

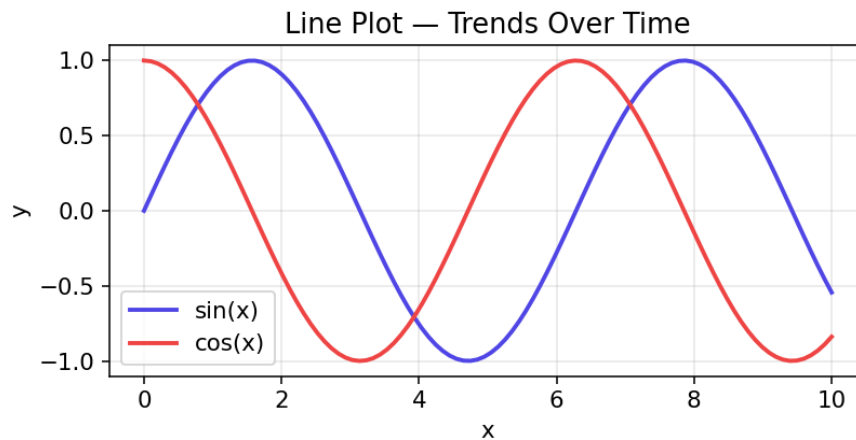


Fig 1 — A line plot showing two curves with a legend.

```
import numpy as np
x = np.linspace(0, 10, 100) # 100 points from 0 to 10

fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(x, np.sin(x), color='blue', linewidth=2, label='sin(x)')
ax.plot(x, np.cos(x), color='red', linewidth=2, label='cos(x)')
ax.set_title('Line Plot')
ax.set_xlabel('x'); ax.set_ylabel('y')
ax.legend() # show the small color box
ax.grid(alpha=0.3) # light grid lines
```

## 4. Bar Plot — Compare Groups

A **bar plot** uses bars of different heights to compare groups. Taller bar = bigger value. Best for comparing categories (■■■■■■■■■■ — ■■■■■ ■■■■■ ■■■), like scores of different cities.

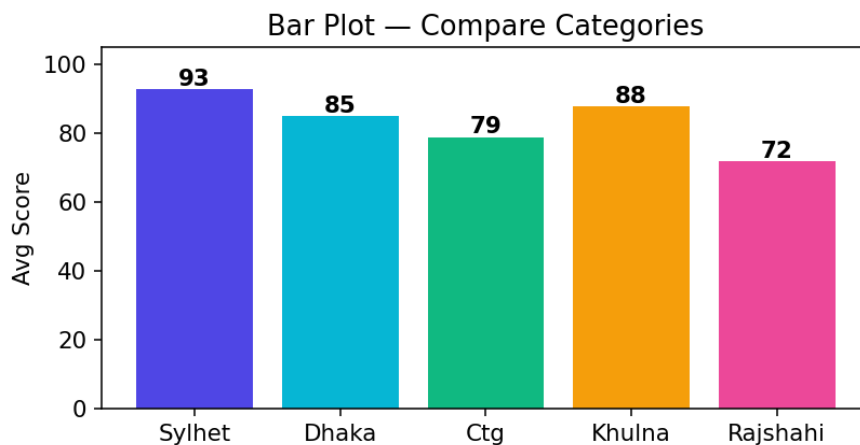


Fig 2 — A bar plot comparing average scores of 5 cities.

```
fig, ax = plt.subplots(figsize=(6, 3))
ax.bar(cities, scores, color='teal')
ax.set_title('Average Score by City')
ax.set_ylabel('Score')

# Horizontal bars instead? use barh:
ax.barh(cities, scores)
```

A **scatter plot** puts one dot for each item, using two values (one for the x position, one for the y position). It shows if two things are related (■■■■■■■■■■). For example: do students who study more get higher marks? If the dots go up together, the answer is yes.



A **histogram** (■■■■■■■■■■) shows how your values are spread out. It puts values into groups called **bins** (■■■ — ■■■■ ■■■ ■■■ ■■■) and counts how many fall in each bin. It answers: are most values small, big, or in the middle?

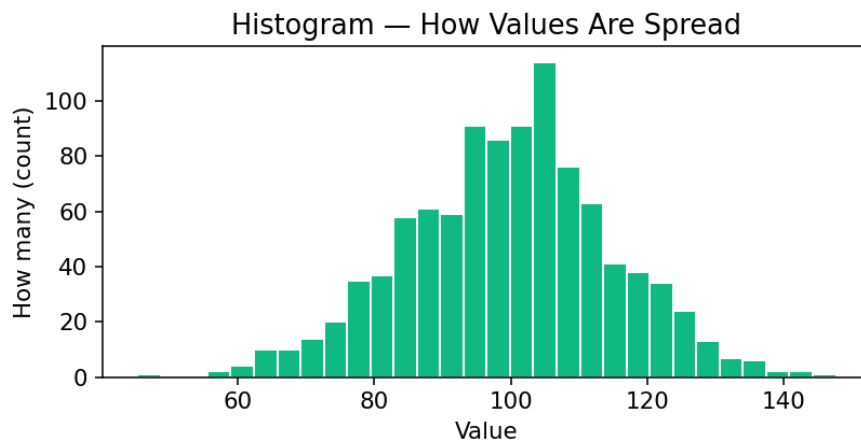


Fig 4 — A histogram. The tall middle shows most values are near 100.

```
fig, ax = plt.subplots(figsize=(6, 3))
ax.hist(data,
        bins=30,          # number of groups
        color='green',
        edgecolor='white') # white line between bars
ax.set_xlabel('Value')
ax.set_ylabel('Count')
```

■ *Histogram vs Bar plot — people mix these up. Bar plot compares separate groups (cities). Histogram shows the spread of ONE number column (ages). They look similar but mean different things.*

## 7. Making Charts Pretty

Small touches make a big difference. Here are the common ones:

```
ax.set_title('Title', fontsize=14, fontweight='bold')
ax.set_xlabel('X label')
ax.set_ylabel('Y label')
ax.legend(loc='upper right') # where to put the legend
ax.grid(True, alpha=0.3)    # alpha = how see-through
ax.set_xlim(0, 100)         # zoom x-axis from 0 to 100
ax.set_ylim(0, 50)          # zoom y-axis
ax.tick_params(labelsize=9) # size of axis numbers

# Colors can be names or hex codes
ax.plot(x, y, color='#4f46e5') # hex code
ax.plot(x, y, color='tomato')   # name

# Line styles
ax.plot(x, y, linestyle='--')  # dashed
ax.plot(x, y, linestyle=':')   # dotted
ax.plot(x, y, marker='o')      # add round dots on points
```

## 8. Many Charts at Once (Subplots)

Sometimes you want several charts side by side on one page. **Subplots** (————) let you make a grid of charts. You tell it how many rows and columns of charts you want.

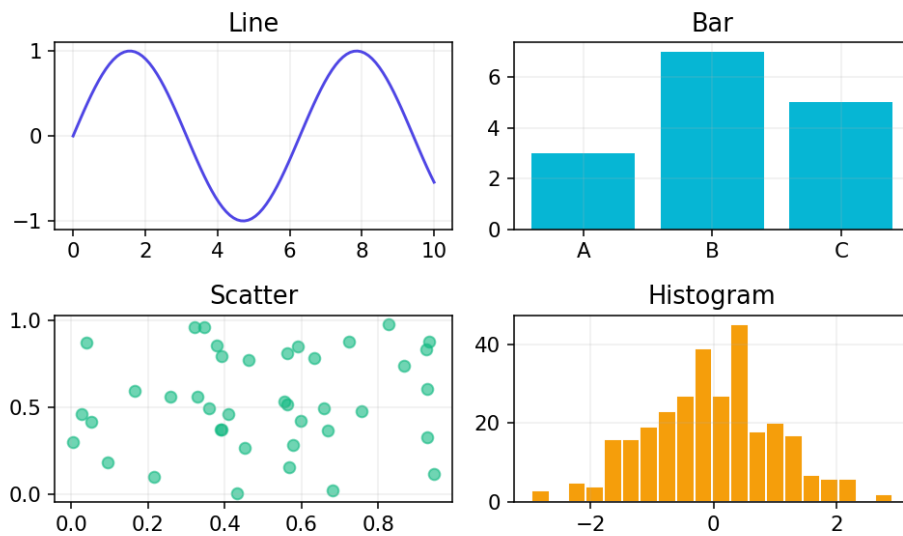


Fig 5 — A 2×2 grid: four different charts on one page.

```
# Make a grid: 2 rows, 2 columns = 4 charts
fig, axs = plt.subplots(2, 2, figsize=(8, 5))

axs[0, 0].plot(x, y)           # top-left
axs[0, 1].bar(cats, vals)      # top-right
axs[1, 0].scatter(a, b)        # bottom-left
axs[1, 1].hist(data)           # bottom-right

axs[0, 0].set_title('Line')    # title each one separately
plt.tight_layout()              # auto-fix spacing (no overlap)
```

## 9. Seaborn — Pretty Charts, Less Code

**Seaborn** does the same charts as Matplotlib but with two big wins: the charts look nicer by default, and you write much less code. Best of all, you just hand it your whole Pandas table and tell it which columns to use.

```
import seaborn as sns
sns.set_theme(style='whitegrid') # nice default look

# Matplotlib way (more code):
ax.bar(df['city'], df['score'])

# Seaborn way (cleaner, works straight from the table):
sns.barplot(data=df, x='city', y='score')

# Common Seaborn charts:
sns.histplot(data=df, x='age') # histogram
sns.scatterplot(data=df, x='hours', y='marks')
sns.lineplot(data=df, x='day', y='sales')
sns.boxplot(data=df, x='day', y='score')
sns.heatmap(df.corr(), annot=True)
```

■ Seaborn and Matplotlib work together. You can make a chart with Seaborn, then use Matplotlib commands like `ax.set_title()` to fine-tune it.

## 10. Boxplot & Heatmap — Find Patterns

### Boxplot — The Spread in One Picture

A **boxplot** (■■■■■■■■■■) shows a lot in a small space. The box shows the middle half of your data. The line inside the box is the **median** (■■■■■■■ — ■■■■■■ ■■■■). The dots far away are **outliers** (■■■■■■■■■■ — ■■■■■■■■■■ ■■■■, ■■■■ ■■■■ ■■ ■■■■ ■■■■).

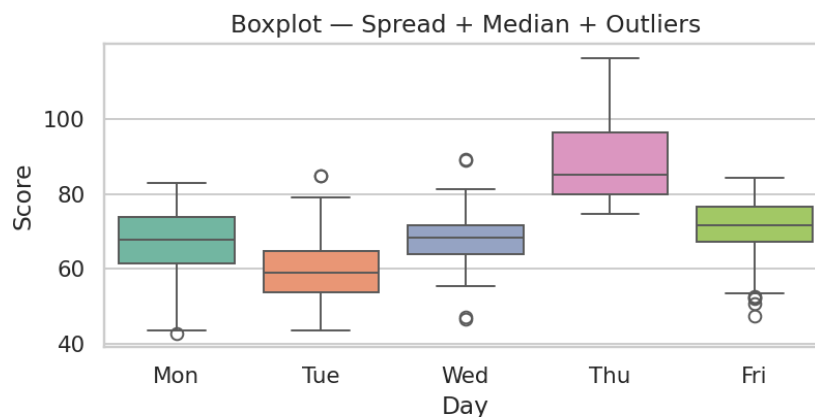


Fig 6 — Boxplots for 5 days. Each box shows that day's spread, median, and outliers.

```
sns.boxplot(data=df, x='Day', y='Score')
```

### Heatmap — Colors Show Strength

A **heatmap** (■■■■■■■■■■) uses color to show numbers. Dark/strong color = big number, light color = small number. The most common use is a **correlation** (■■■■■■■■■■ — ■■■■ ■■■■■■ ■■■■■■ ■■■■■■ ■■■■■■) map, to see which features move together. This is super useful in ML feature selection.



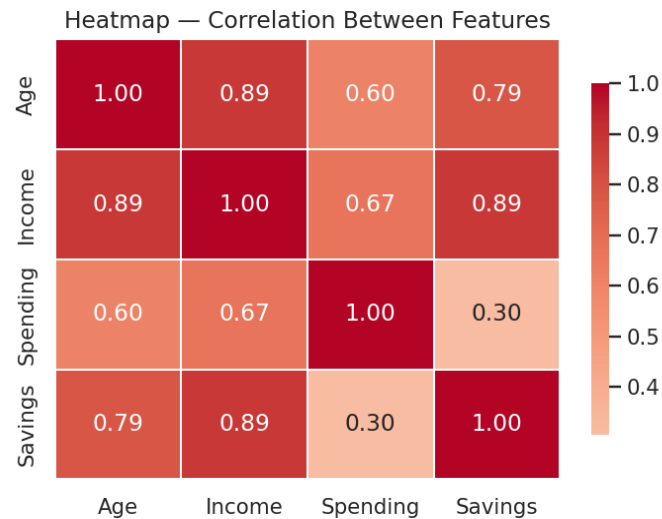


Fig 7 — Correlation heatmap. Values near 1 (red) = strong link; near 0 = no link.

```
# corr() gives a table of how columns relate (-1 to 1)
sns.heatmap(df.corr(),
             annot=True,      # write the number in each box
             cmap='coolwarm', # color scheme
             center=0,        # 0 sits in the middle color
             fmt='.2f')       # 2 decimal places
```

## 11. Which Chart Should I Use?

Picking the right chart is half the job. Quick guide:

| I want to...              | Use this chart | Function              |
|---------------------------|----------------|-----------------------|
| Show change over time     | Line plot      | plot / lineplot       |
| Compare separate groups   | Bar plot       | bar / barplot         |
| See if two things relate  | Scatter plot   | scatter / scatterplot |
| See spread of one column  | Histogram      | hist / histplot       |
| See median + outliers     | Boxplot        | boxplot               |
| Count items per category  | Countplot      | countplot             |
| See which features relate | Heatmap        | heatmap               |
| See all pairs at once     | Pairplot       | pairplot              |

## 12. A Real EDA Example

Here is how you actually use these tools when you start a new ML project. After loading and cleaning data with Pandas, you draw charts to understand it.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style='whitegrid')

df = pd.read_csv('titanic.csv')

# 1. How are ages spread out?
sns.histplot(data=df, x='Age', bins=30)
plt.show()

# 2. Did more women survive than men?
sns.countplot(data=df, x='Sex', hue='Survived')
plt.show()

# 3. Which numeric features relate to each other?
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()

# 4. Did ticket class affect age/fare?
sns.boxplot(data=df, x='Pclass', y='Fare')
plt.show()

```

■ *This is real work. Every chart answers one question about your data. By the end you understand your dataset — and that helps you build a better model.*

## 13. Common Mistakes

| Mistake                   | What happens               | Fix                            |
|---------------------------|----------------------------|--------------------------------|
| Forgetting plt.show()     | Chart never appears        | Call plt.show() at the end     |
| Charts overlap each other | Messy, unreadable          | Use plt.tight_layout()         |
| No title or labels        | Nobody knows what it means | Always add title + axis labels |
| Too many colors           | Confusing rainbow          | Use 2-4 clear colors           |
| Wrong chart type          | Hides the real pattern     | Check the 'which chart' guide  |
| Reusing same figure       | New plot draws on old one  | Make a fresh fig each time     |

## Quick Reference Cheatsheet

---

| Task              | Code                                                  |
|-------------------|-------------------------------------------------------|
| Start a chart     | <code>fig, ax = plt.subplots(figsize=(6,4))</code>    |
| Line plot         | <code>ax.plot(x, y)</code>                            |
| Bar plot          | <code>ax.bar(cats, vals)</code>                       |
| Horizontal bars   | <code>ax.barh(cats, vals)</code>                      |
| Scatter plot      | <code>ax.scatter(x, y)</code>                         |
| Histogram         | <code>ax.hist(data, bins=30)</code>                   |
| Title             | <code>ax.set_title('Name')</code>                     |
| Axis labels       | <code>ax.set_xlabel('..'); ax.set_ylabel('..')</code> |
| Legend            | <code>ax.legend()</code>                              |
| Grid              | <code>ax.grid(alpha=0.3)</code>                       |
| Zoom axis         | <code>ax.set_xlim(0,100)</code>                       |
| Grid of charts    | <code>plt.subplots(2, 2)</code>                       |
| Fix spacing       | <code>plt.tight_layout()</code>                       |
| Show chart        | <code>plt.show()</code>                               |
| Save chart        | <code>plt.savefig('f.png', dpi=150)</code>            |
| Seaborn theme     | <code>sns.set_theme(style='whitegrid')</code>         |
| Seaborn bar       | <code>sns.barplot(data=df, x='a', y='b')</code>       |
| Seaborn hist      | <code>sns.histplot(data=df, x='a')</code>             |
| Seaborn scatter   | <code>sns.scatterplot(data=df, x='a', y='b')</code>   |
| Seaborn boxplot   | <code>sns.boxplot(data=df, x='a', y='b')</code>       |
| Seaborn countplot | <code>sns.countplot(data=df, x='a')</code>            |
| Seaborn heatmap   | <code>sns.heatmap(df.corr(), annot=True)</code>       |
| Seaborn pairplot  | <code>sns.pairplot(df)</code>                         |

**Next Topic:** SQL & PostgreSQL — how to store data in a database and pull out exactly what you need using queries. The skill that feeds your Pandas tables.